

Low-Light Image Enhancement with Cross-Dataset Generalization

Cosimo Curri
Politecnico di Bari
Bari, Italy

c.curri@studenti.poliba.it

Abstract

...

1. Introduction

...

2. Datasets and Splits

2.1. Datasets

The pipeline developed in this work is based on four dataset sources, each with a specific role in the experimental protocol: (i) LOL-v2 [9], (ii) LOL-v1 [8], (iii) MILLS [5], and (iv) ExDark [1]. All models are trained on a single source domain and are then evaluated on additional datasets without fine-tuning. This choice is intended to measure cross-dataset generalization, i.e., the ability of each model to enhance low-light images whose distribution differs from the training data.

The main training dataset is LOL-v2. This dataset contains both a real-captured subset, hereafter referred to as LOL-v2 Real, and a synthetic subset, referred to as LOL-v2 Synthetic. In the main protocol, only LOL-v2 Real is used, meaning that the low-light/normal-light pairs are acquired from real scenes rather than obtained through synthetic low-light degradation. LOL-v2 is a paired dataset: each low-light image is associated with a corresponding normally exposed reference image, which can be used as the target for supervised training and evaluation.

LOL-v1 is used only as a paired cross-dataset evaluation set. It represents a relatively close cross-dataset scenario, because it belongs to the same general benchmark family as LOL-v2.

MILLS is used as a second paired cross-dataset evaluation set. In particular, the low-resolution DSLR split, MILLS-DSLR, is adopted to keep the evaluation computationally manageable. Unlike LOL-v1, this dataset introduces a controlled illumination setting. Its input images are associated

with different illumination levels, and each input has a corresponding well-illuminated reference image. This makes it possible to evaluate the models not only with global results over the whole dataset, but also by studying how performance changes as the input illumination becomes more severe.

Finally, ExDark is used as an unpaired stress-test dataset. Here, unpaired means that the dataset provides low-light images without a corresponding normally exposed reference image. Therefore, ExDark is not used for reference-based comparison, but for no-reference evaluation and qualitative failure analysis on real-world dark scenes. Its category-based organization is preserved, meaning that images remain grouped according to the object category provided by the dataset, such as cars, people, animals, or indoor objects. This organization may help inspect whether some failure cases are related to the type of scene or object.

2.2. Dataset Licenses and Terms of Use

Since this project relies on publicly available datasets, their licenses and terms of use were reviewed to ensure that each dataset is used consistently with the conditions specified by its original authors.

For LOL-v1¹ and LOL-v2², no explicit dataset license or terms of use were found in the official sources consulted and linked in this paper.

MILLS³ is distributed through its Hugging Face dataset page, where the dataset metadata reports the license as MIT. This permissive license allows use, modification, and redistribution, provided that the copyright notice and permission notice are included in copies or substantial portions of the material.

ExDark⁴ is subject to dataset-specific usage restrictions. Although the repository reports the project as open source under the BSD-3 license, it also states that commercial usage requires contacting the authors. More specifically, the dataset

¹Official Source for LOL-v1

²Official Source for LOL-v2

³Official Source for MILLS

⁴Official Source for ExDark

README indicates that the ExDark dataset may be used only for non-commercial research purposes.

All datasets are used exclusively within the scope of this non-commercial academic work. Any applicable license terms or usage restrictions are respected, and the corresponding dataset papers are cited in the bibliography.

To make these conditions explicit, a dedicated dataset notice is provided both in the project repository and in the accompanying release. Any dataset files used or redistributed with this project remain subject to the original authors’ licenses and terms of use, and are provided solely to support non-commercial academic reproducibility.

2.3. Split Construction

Split construction follows the role assigned to each dataset, as discussed in Section 2.1. For the main source domain, LOL-v2 Real, the official training split is used to derive the training and validation partitions. In particular, a fixed fraction of the official training samples is deterministically held out as validation data using a fixed random seed. For this project, this held-out fraction is set to 10% of the official training samples by the configuration file of the pipeline stage described in Section 4.1. The official test split is kept untouched for the final in-domain evaluation. In this context, “in-domain” means that training, validation, and testing images come from the same dataset and acquisition setting.

The synthetic subset of LOL-v2 is kept separate from the main protocol and is used only for an additional ablation study, aimed at evaluating whether synthetically degraded low-light examples improve or degrade generalization. When this subset is used, a validation partition is deterministically derived from the official training split using the same procedure adopted for LOL-v2 Real, ensuring consistency in the split construction process.

For the remaining datasets, no additional train or validation partitions are created. Instead, all available samples are assigned to a single evaluation partition. For datasets that already provide predefined splits, such as LOL-v1 and MILLS, the original partitions are merged into a unified test set. ExDark is likewise converted into a single test split, while preserving the original category labels as metadata.

Following the split construction procedure, the resulting split sizes are summarized in Table 1.

3. Experimental Protocol

The experimental protocol is designed to evaluate both in-domain performance and cross-dataset generalization while preventing test leakage. All models are trained on a single source domain, LOL-v2 Real, and the validation split is used for early stopping, checkpoint selection, and model selection. The test sets are never used during these steps and are evaluated only after the final configuration has been selected.

Table 1. Dataset split sizes used in the pipeline. Counts refer to samples, i.e., low-light input images. For paired datasets, each input sample is associated with a corresponding target image.

Dataset	Train	Validation	Test
ExDark	–	–	7363
LOL-v1	–	–	500
LOL-v2 Real	620	69	100
LOL-v2 Synthetic	810	90	100
MILLS	–	–	499

The project is organized around model families. For each family, a dedicated grid search explores meaningful architectural and optimization choices, while the other parts of the experimental procedure are kept unchanged whenever possible. These common parts include the data augmentation policy, the training and validation procedure, and the validation-based selection strategy. In this way, different model families can be compared under consistent experimental conditions and under a similar computational budget.

Across comparable runs, the training setup keeps fixed the batch size, the data-loading strategy, the validation frequency, the use of mixed precision when supported by the hardware, and the choice of favoring efficiency over fully deterministic execution. The maximum number of training epochs is intentionally set as a large upper bound, because the effective training duration is controlled by early stopping rather than by a fixed epoch budget.

Early stopping is implemented in a model-agnostic way. The monitored validation metric, its optimization direction, and the optional tie-breaker metric can be selected according to the specific experiment or model family. However, the overall early-stopping policy is kept fixed across comparable runs, so that all configurations are selected according to the same validation-based logic.

For each model family, the experimental protocol is organized into the following stages: (i) a screening stage, in which all configurations of the grid search are trained with a fixed seed and ranked using validation performance; (ii) a multi-seed re-training stage, in which the most promising configurations are re-trained with multiple random seeds to estimate both performance and stability; (iii) a validation-based model selection stage, in which the final configuration is chosen from the aggregated multi-seed validation results; and (iv) a final evaluation stage, in which the selected configuration is tested on the in-domain test set and on all cross-dataset test sets. During screening and model selection, test evaluation is disabled; it is enabled only in the final evaluation stage. Figure 1 provides a visual overview of this workflow, while the following subsections describe each stage in detail.

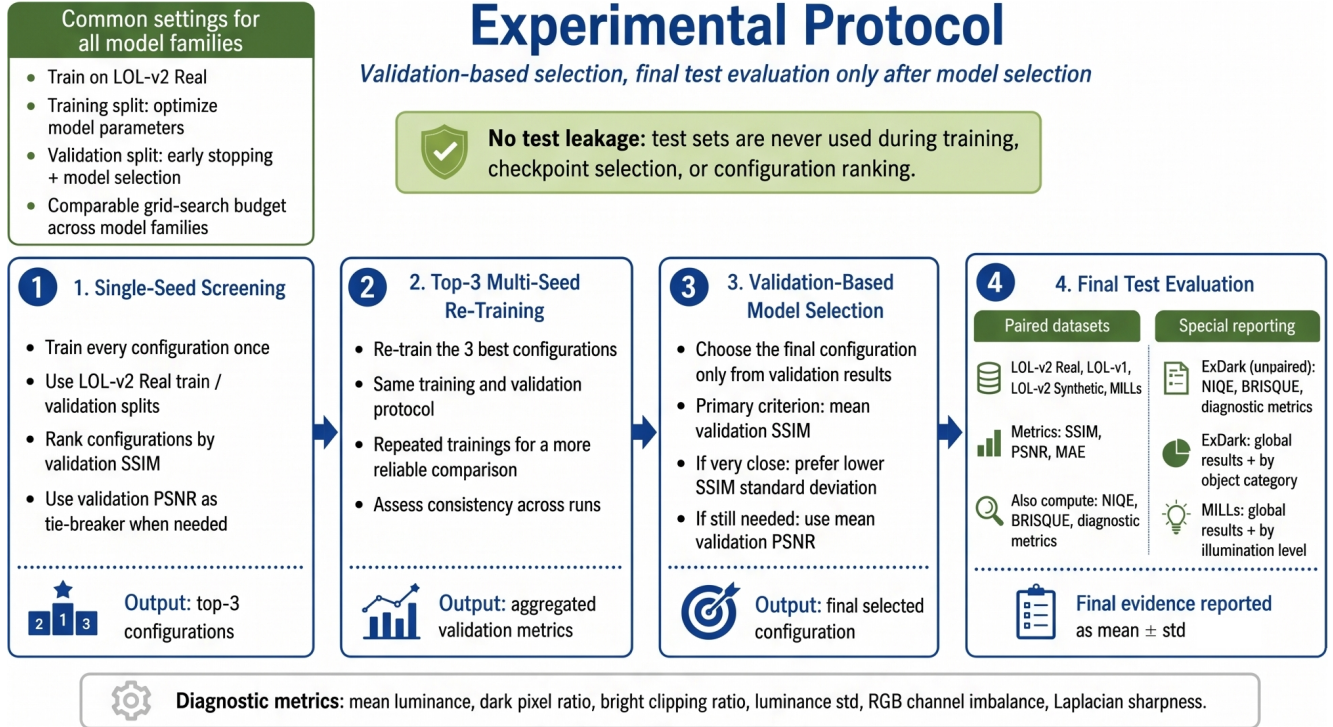


Figure 1. Overview of the experimental protocol.

3.1. Single-Seed Screening

The first experimental stage is a single-seed grid search, in which every configuration is trained using a single fixed seed. The purpose of this stage is not to produce final evidence, because a single seed can be affected by random initialization and data-loader randomness. Instead, the purpose is to reduce the search space and identify a small number of promising configurations.

Each configuration is trained on the LOL-v2 Real training split and monitored on the LOL-v2 Real validation split. The test sets are not evaluated during screening. For each run, validation structural similarity index measure (SSIM) [7] is used as the primary criterion for checkpoint selection. This choice reflects the goal of low-light enhancement: the output should remain structurally similar to the reference image, preserving luminance, contrast, and local image structure, rather than only reduce the average pixel-wise error. Validation peak signal-to-noise ratio (PSNR) is also computed and used as a tie-breaker because it measures pixel-wise reconstruction quality in logarithmic form. Therefore, when two checkpoints achieve validation SSIM values within the configured tolerance, PSNR is used to select the best checkpoint for that run.

The output of this stage is a validation-based ranking of the explored configurations. This ranking is later parsed from the run logs and used to select the top-3 configurations

for the multi-seed stage.

3.2. Top-3 Multi-Seed Re-Training

After the screening stage, the three most promising configurations are selected from the validation-based ranking and re-trained under the same training and validation protocol. This stage is necessary because the screening phase provides only a preliminary estimate of performance, obtained from a single training run for each configuration.

The goal of the multi-seed stage is to obtain a more reliable comparison among the selected configurations. By repeating the training procedure several times for each of them, the protocol can evaluate whether its validation performance is consistent or mainly due to a favorable run.

The outputs of this stage are aggregated validation metrics for each selected configuration. These aggregated results are then used for the final validation-based selection, while the test sets remain untouched.

3.3. Validation-Based Model Selection

The final configuration is selected only from the aggregated validation results of the multi-seed stage.

The primary selection criterion is the mean validation SSIM across runs. When configurations obtain very close mean SSIM values, the selection favors the more stable configuration, measured by the standard deviation of validation SSIM. If the comparison is still ambiguous, mean validation

PSNR is used as a final tie-breaker.

This rule keeps the whole selection process validation-based, while also accounting for statistical robustness across repeated trainings.

3.4. Final Test Evaluation

The final test evaluation is performed only after the final configuration has been selected. The selected configuration is evaluated on the test set of all the datasets used in this project, and the resulting metrics are aggregated as mean and standard deviation. This ensures that the final evidence is not based on a single run, but reflects both average performance and statistical robustness.

The evaluation protocol depends on whether a dataset is paired or unpaired. In paired datasets, “full-reference” metrics can be computed by comparing the enhanced output of the model with the corresponding target image of the dataset. In this work, paired datasets are evaluated using SSIM, PSNR, and mean absolute error (MAE). This last metric measures the average absolute pixel difference between the enhanced image and the reference image.

“No-reference” metrics do not require target images and are computed for both paired and unpaired datasets. In this work, the no-reference metrics are naturalness image quality evaluator (NIQE) [3] and blind/referenceless image spatial quality evaluator (BRISQUE) [2]. More specifically, NIQE measures the deviation of an image from statistical properties commonly observed in natural images, while BRISQUE estimates perceptual quality from spatial scene statistics.

Additional diagnostic metrics are computed on the model output for both paired and unpaired datasets. They are used to support the interpretation of failure modes, meaning recurring types of enhancement errors. These diagnostics include: (i) mean luminance, which measures the average brightness of the enhanced image; (ii) dark pixel ratio, which measures the fraction of pixels that remain very dark; (iii) bright clipping ratio, which measures the fraction of pixels close to saturation; (iv) luminance standard deviation, which provides a compact indication of contrast variation; (v) red-green-blue (RGB) channel imbalance, which helps detect color cast; and (vi) Laplacian sharpness, which provides a simple estimate of high-frequency content related to blur, noise, or halo artifacts.

LOL-v1 and LOL-v2 are paired datasets and are therefore evaluated by aggregating full-reference metrics, no-reference metrics, and diagnostic output metrics over their respective test sets. ExDark and MILs require a more specialized reporting protocol because of their particular dataset structure and evaluation objectives.

For ExDark, which is unpaired, no normally exposed target image is available. Therefore, it is evaluated using only no-reference metrics and diagnostic output metrics. In addition to global results over the whole dataset, ExDark

is also analyzed by object category, since its images are organized into semantic groups.

MILs is a paired dataset and is therefore evaluated similarly to LOL-v1 and LOL-v2. However, its results are reported at two levels: globally across all samples and separately by illumination level. This grouped reporting is particularly useful because MILs includes inputs captured under different illumination conditions, enabling a more detailed analysis of how model performance varies with illumination.

4. End-to-End Pipeline

The pipeline connects the datasets and splits discussed in Section 2 with the experimental protocol defined in Section 3. Its role is to transform raw datasets into a common representation and then use that representation to train, select, evaluate, and analyze the models in a reproducible way.

The implemented workflow is organized into five main stages: (i) manifest generation, where raw dataset folders are converted into comma-separated values (CSV) manifests; (ii) offline preprocessing, where images are deterministically processed once, updated manifests are written, and dataset-level statistics are computed; (iii) experiment execution, where configuration files are expanded into concrete runs and the framework performs online data augmentation, training, validation-based model selection, checkpointing, logging, and optional test evaluation; (iv) log parsing and aggregation, where the resulting JSON Lines (JSONL) logs are used to build rankings and aggregate validation or test metrics; and (v) qualitative example export, where selected checkpoints are restored to produce visual examples for inspection.

Figure 2 summarizes the implemented workflow and shows how the dataset preparation stages are connected to experiment execution, log aggregation, and qualitative analysis.

“Offline” preprocessing and “online” augmentation serve different purposes in the pipeline. Offline preprocessing consists of deterministic operations that are performed once before training, saved on disk, and kept fixed across experiments. Online augmentation is instead applied at training time, when samples are loaded from the training split, and has a stochastic nature because the transformations are randomly sampled during training. In the current configuration, geometric augmentations are applied consistently to both the low-light input and the paired target, while photometric perturbations are applied only to the input image. This preserves the alignment of paired samples while exposing the model to additional low-light variations.

The pipeline follows a central design principle of the project: only the manifest generation stage is intentionally dataset-specific. This stage absorbs the differences between the original dataset structures, such as pairing rules, filename

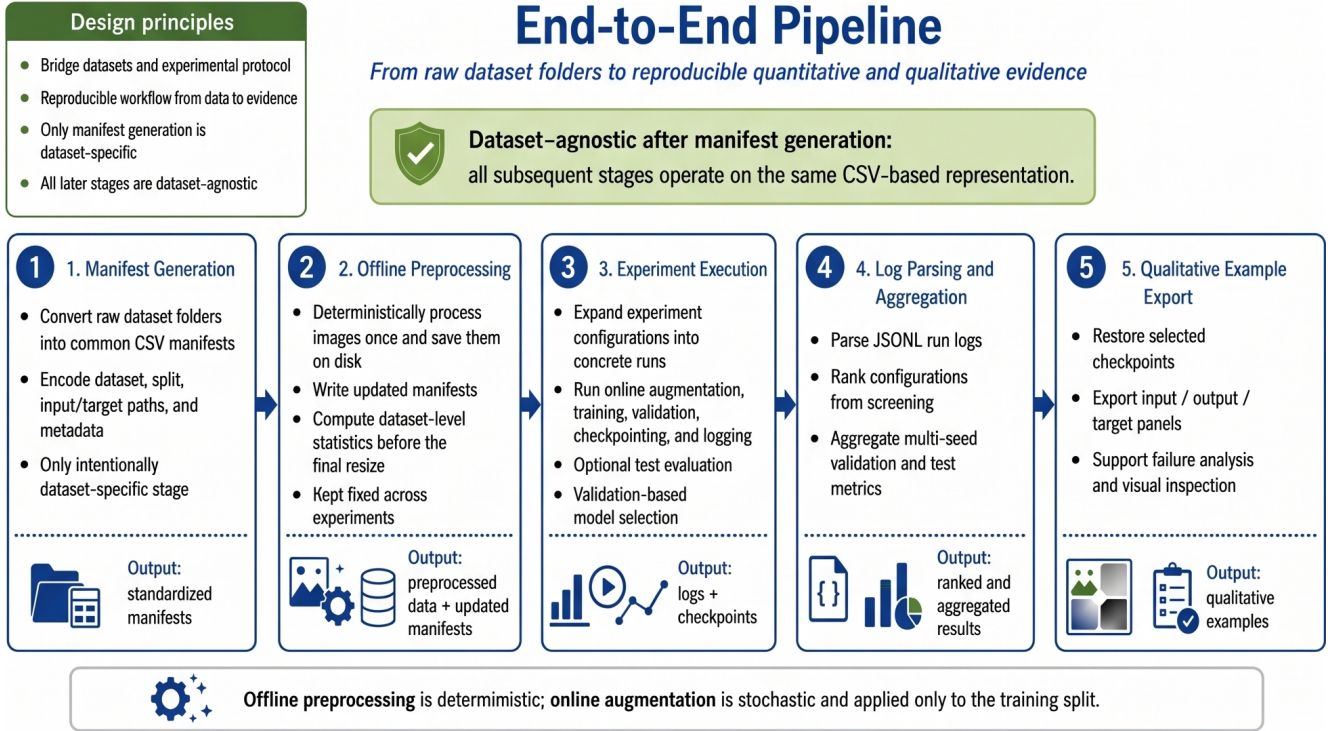


Figure 2. Overview of the end-to-end pipeline.

conventions, category folders, and illumination-level metadata. Once each dataset has been converted into the CSV manifest format, the following stages of the pipeline operate on the same representation and do not need to know how each dataset was originally organized on disk.

The following subsections describe these stages in more detail, showing how each part of the pipeline is implemented in the codebase.

4.1. Manifest Generation

The purpose of the first stage of the pipeline is to scan the original dataset folders and build CSV manifest files from their structure. In each manifest, every row represents one image sample and stores the dataset name, the split to which the sample belongs, the path of the low-light input image, the path of the corresponding target image when available, and optional metadata such as the image category or the illumination level.

This step is necessary because the datasets used in this project are not organized in the same way. Some datasets provide paired low-light and normal-light images with the same filename, while others use different filename conventions. ExDark is instead unpaired and organized by object category. MILs also includes an illumination level in the input filename, which is useful for analyzing how performance changes under different lighting conditions. Without this standardization step, the subsequent scripts in the pipeline

would need to contain several dataset-specific rules.

For this reason, manifest generation is the only intentionally dataset-specific part of the pipeline. The script reads a YAML configuration file that describes where each dataset is located, which output file should be produced, and how the corresponding manifest should be created. In LOL-v1 and LOL-v2 Synthetic, low-light images and their corresponding reference images share the same filename, making the association straightforward. In LOL-v2 Real, the filenames differ only because low-light and reference images use different prefixes. ExDark follows a completely different structure: images are grouped by category and no reference image is provided, so only the image path and category information are recorded. Finally, in MILs, the filename also contains information about the illumination level, which is extracted and stored together with the path of the corresponding reference image.

The output of this step is a CSV manifest file for each dataset, with a fixed column structure. This common format is what makes the later stages dataset-agnostic: once the manifests have been created, preprocessing, training, and evaluation only need to read the paths and metadata stored in the CSV files, without knowing how the original dataset was arranged on disk.

The manifest generation script also supports reproducibility-oriented choices. When required by the protocol and when a training split is available, a

validation split can be sampled from the training split using a fixed random seed. Moreover, the generated rows are sorted according to the split and image path, making the row order deterministic across runs.

The same manifest information is also useful during evaluation. For ExDark, the object category associated with each image makes it possible to report results not only globally, but also separately for each category. For MILs, the illumination level associated with each image makes it possible to report results both over the whole dataset and separately for each illumination level. In this way, the evaluation can remain general while still preserving the specific information needed for each dataset.

4.2. Offline Preprocessing

Deterministic preprocessing. After manifest generation, all datasets are processed by an offline preprocessing script. This stage is called “offline” because it contains deterministic preliminary operations that are performed once for all the experiments of this project rather than repeatedly during training or evaluation. The processed images are stored on disk and are then used by all later stages of the pipeline.

For each CSV manifest, the script first performs a lightweight consistency check. It verifies that no input path is duplicated within the same split and that the same input path does not appear in different splits. These checks are intentionally limited, since the dataset-specific pairing rules have already been handled during manifest generation.

After these checks, the script creates a preprocessed version of each dataset on disk. For each dataset, it also writes a new CSV manifest with the same columns as the original one, but with paths updated to the preprocessed images. The output paths preserve the dataset-relative folder structure of the original files, while the image extension is replaced according to the selected output format.

Each image undergoes the following deterministic operations: (i) loading with Exchangeable Image File Format (EXIF) orientation applied when available, (ii) conversion to RGB, (iii) resizing to 256×256 using Lanczos interpolation, and (iv) saving in Portable Network Graphics (PNG) format. Applying the EXIF orientation avoids processing photographs with an incorrect rotation. PNG is used to avoid introducing additional compression artifacts during preprocessing. If the same source image is referenced more than once in a manifest, it is processed only once and then reused, avoiding possible duplicated work.

A fixed resizing strategy is used instead of cropping. In this project, preserving the whole scene during preprocessing was considered preferable to selecting a local region, because a crop would permanently discard part of the image and could remove relevant low-light regions, light sources, or objects needed for cross-dataset analysis. Although resizing may slightly distort the aspect ratio of some images, it keeps

the full scene and provides a fixed deterministic input size for all datasets. This choice does not exclude the use of cropping as online data augmentation during training: unlike offline cropping, random resized crops are stochastic, do not permanently modify the preprocessed dataset, and are applied only to the training split, while validation and test images remain deterministic.

Dataset statistics. In addition to producing the preprocessed images and updated manifests, the script computes simple dataset-level statistics and writes them to a dedicated summary file. These statistics are computed before applying the final resize, after loading each image with the correct orientation and converting it to RGB. Therefore, they summarize the original image sizes and the illumination properties of the images before they are converted into the fixed 256×256 representation used as model input.

For each image of size $H \times W$, a luminance map $Y \in [0, 1]^{H \times W}$ is computed from the RGB channels using the ITU-R BT.709 luma coefficients:

$$Y_{i,j} = 0.2126R_{i,j} + 0.7152G_{i,j} + 0.0722B_{i,j}, \quad (1)$$

where $R_{i,j}$, $G_{i,j}$, and $B_{i,j}$ are the RGB values of pixel (i, j) , scaled to $[0, 1]$. Based on this luminance map, the dark pixel ratio is defined as

$$\text{DPR} = \frac{1}{HW} \sum_{i=1}^H \sum_{j=1}^W 1[Y_{i,j} < \tau_d], \quad \tau_d = 0.10, \quad (2)$$

where H and W are the image height and width respectively, τ_d is the dark-pixel threshold, and $1[\cdot]$ is the indicator function, equal to one when the condition is true and zero otherwise.

For each dataset, the script records the number of samples, the median original input size, the mean and standard deviation of input luminance, and the average fraction of dark pixels. For paired datasets, the same luminance summary is also computed on the target images. These statistics are written to a dataset summary file and are reported in Table 2.

The statistics in Table 2 provide a compact description of the illumination distribution encountered by the preprocessing pipeline. LOL-v1 and LOL-v2 Real have very similar input luminance distributions, supporting the use of LOL-v1 as a close cross-dataset benchmark for the main training domain. LOL-v2 Synthetic and MILs are instead brighter on average, indicating a larger distribution shift in terms of illumination. ExDark lies between these cases in terms of input luminance and remains useful as an unpaired real-world stress test.

4.3. Experiment Execution

Run configuration. After offline preprocessing, the third stage of the pipeline is experiment execution. This stage uses

Table 2. Dataset statistics computed during offline preprocessing, before applying the final resize. Luminance values are reported as mean $\pm$ standard deviation across samples. Dark pixels denote the average fraction of input pixels with luminance below 0.10.

Dataset	Samples	Median original size	Input luminance	Dark pixels	Target luminance
ExDark	7363	640 $\times$ 500	0.143 $\pm$ 0.086	59.6%	–
LOL-v1	500	600 $\times$ 400	0.061 $\pm$ 0.041	82.7%	0.458 $\pm$ 0.102
LOL-v2 Real	789	600 $\times$ 400	0.062 $\pm$ 0.042	80.9%	0.436 $\pm$ 0.105
LOL-v2 Synthetic	1000	384 $\times$ 384	0.240 $\pm$ 0.138	46.7%	0.513 $\pm$ 0.141
MILLs	499	600 $\times$ 400	0.244 $\pm$ 0.110	16.4%	0.369 $\pm$ 0.095

the preprocessed manifests to run the experiments required by the protocol. It is model-agnostic: the pipeline does not assume a specific architecture. Instead, each run is built from an experiment configuration, which specifies the model family, the training and validation manifests, the hyperparameters, the loss function, the online data augmentation policy, the early-stopping rule, and whether test evaluation should be executed.

The experiment launcher expands the hyperparameter grid defined in the configuration and creates one resolved configuration for each combination of values, i.e., a set of configurations in which every grid parameter has been replaced by one concrete value. This makes it possible to describe a large set of experiments compactly.

Each resolved configuration is assigned a configuration identifier. This identifier is computed from the part of the configuration that defines the model and its training setup, while run-specific or evaluation-specific fields are ignored. In this way, repeated runs of the same configuration, for example with different seeds, can be recognized and aggregated correctly.

Training, data augmentation, and validation. For each run, the framework builds the model, the loss function, the optimizer, and the data loaders from the resolved configuration. Model-specific details are isolated behind lightweight model-family wrappers, which define how the model and loss are constructed, how a batch is passed through the model, and how the model output is interpreted as a prediction. Training and validation samples are read from the preprocessed manifests, using the split field to distinguish the two partitions. Since at this point of the pipeline the input format is the same for all datasets, the execution stage no longer depends on the original dataset folder structure.

Online data augmentation is also part of experiment execution. The augmentation policy is defined in the experiment configuration and can enable or disable each transformation and control its sampling behavior. In the current experimental protocol, however, this policy is kept fixed across runs, so that model configurations are compared under the same data variability.

Augmentation is applied only to samples loaded from the

training split. The framework explicitly prevents it from being enabled for validation or test splits, so validation and test metrics are always computed on deterministic preprocessed images. The augmentation pipeline is applied after reading the preprocessed RGB images and before converting them to tensors.

The transformations are considered in a fixed order: (i) horizontal flipping, (ii) random resized cropping, (iii) color jitter, (iv) gamma jitter, and (v) additive synthetic sensor noise. When enabled, random resized cropping is applied to each training sample by randomly sampling the crop parameters. The other stochastic transformations are applied according to their configured probabilities. Therefore, a training sample does not always receive the full sequence of transformations: depending on the random decisions, different samples can undergo different combinations of augmentations. This creates a wide variety of possible augmented inputs while keeping the augmentation policy fixed across experiments.

Each transformation has a specific role. Horizontal flipping exposes the model to mirrored versions of the same scene. Random resized cropping changes the visible region and slightly varies the scale of the content. Color jitter perturbs brightness, contrast, and saturation, while gamma jitter modifies the brightness curve of the input image, affecting dark and bright regions differently. Finally, synthetic sensor noise adds small random perturbations to simulate noisy low-light acquisitions.

Horizontal flipping and random resized cropping are geometric transformations. They are applied consistently to both the low-light input and the paired target, using the same random decision and the same spatial parameters. This preserves the pixel-level alignment required by supervised training. Color jitter, gamma jitter, and synthetic sensor noise are photometric perturbations. They are applied only to the low-light input, because their purpose is to simulate additional low-light variability without modifying the reference image. This distinction allows the model to see a broader range of degraded inputs while keeping the target image as a stable normally exposed reference.

Training is performed on paired samples. The model receives the low-light input image and predicts an enhanced

RGB image. The loss function then compares this prediction with the corresponding reference image according to the objective specified in the configuration. At the configured validation interval, the framework evaluates the model on the validation split, computes the configured validation metrics, updates the early-stopping state, and stores the model state when a new best validation result is obtained.

Before being passed to the model, training images are converted to tensors after online augmentation, while validation images are converted directly after loading because validation is not augmented. In both cases, image tensors have values in $[0, 1]$. Therefore, the model inputs, the reference targets, the predictions, the paired losses, and the full-reference metrics all use the same normalized image range.

For reproducibility, the paired quantities used during training and validation are defined below. Given a prediction $\hat{I}$ and a target image I , both with C channels and spatial size $H \times W$, the MAE is computed as

$$\text{MAE}(\hat{I}, I) = \frac{1}{CHW} \sum_{c=1}^C \sum_{i=1}^H \sum_{j=1}^W |\hat{I}_{c,i,j} - I_{c,i,j}|. \quad (3)$$

The same expression can also be used as an L1 loss term during training when selected by the configuration.

SSIM measures the structural similarity between the prediction and the reference image. It compares local luminance, contrast, and structure using a local averaging window applied independently to each channel. For each local window, SSIM is defined as

$$\text{SSIM}(\hat{I}, I) = \frac{(2\mu_{\hat{I}}\mu_I + C_1)(2\sigma_{\hat{I}I} + C_2)}{(\mu_{\hat{I}}^2 + \mu_I^2 + C_1)(\sigma_{\hat{I}}^2 + \sigma_I^2 + C_2)}, \quad (4)$$

PSNR is computed from the mean squared error (MSE). The MSE is defined as

$$\text{MSE}(\hat{I}, I) = \frac{1}{CHW} \sum_{c=1}^C \sum_{i=1}^H \sum_{j=1}^W (\hat{I}_{c,i,j} - I_{c,i,j})^2, \quad (5)$$

and PSNR is then computed as

$$\text{PSNR}(\hat{I}, I) = 10 \log_{10} \left(\frac{1}{\max(\text{MSE}(\hat{I}, I), 10^{-10})} \right). \quad (6)$$

The small lower bound inside the maximum avoids numerical instability when the MSE is extremely close to zero.

During validation, the framework computes the loss and the full-reference metrics on the validation split. No-reference and diagnostic metrics are skipped at this stage, because they are not used for checkpoint selection and would make the validation loop heavier.

Early stopping is configured in a model-agnostic way. The framework monitors a primary validation metric and can

use a secondary validation metric as a tie-breaker when the primary metric changes by less than the configured tolerance. When the number of consecutive non-improving validation steps reaches the patience value, training stops.

Mixed precision is controlled by the experiment configuration. When enabled, it allows some training computations to use lower numerical precision, which can reduce memory usage and improve speed. This option is used only when a CUDA device, namely an NVIDIA graphics processing unit (GPU), is available. In that case, the framework also uses a gradient scaler: it temporarily multiplies the loss by a scaling factor before backpropagation, so that very small gradients are easier to represent in lower precision, and then scales the gradients back before the optimizer updates the model weights. This helps reduce numerical instability while still benefiting from mixed precision. If mixed precision is disabled or CUDA is not available, training is performed in standard precision.

Logging and checkpointing. For each run, the framework writes a JSONL log, in which every line is a separate JSON object. Each resolved run is written to a separate JSONL file inside an experiment-specific log directory, so that later parsing scripts can process runs independently and aggregate them by configuration. This format is useful because later parsers can read the logs line by line and extract only the events needed for ranking, aggregation, or reporting. The first log event stores general information about the run, including the resolved configuration, the random seed, the device, the checkpoint location, and the total number of model parameters. More specifically, the number of parameters is computed as the sum of all tensor elements in the model parameters, and it can later be used to compare configurations not only in terms of validation performance, but also in terms of model size.

Each run also saves a checkpoint containing both the model weights and the additional information needed to resume or reuse the run. This file includes the current and best validation model weights, optimizer information, the mixed-precision scaler when applicable, training and early-stopping information, random generator states, and the resolved configuration. Therefore, the same checkpoint can be used to resume training, restore the best validation model for test evaluation, or export qualitative examples.

Test evaluation. When test evaluation is enabled in the experiment configuration file, it is also executed inside this stage of the pipeline. Before evaluation starts, the framework restores the best model state selected on the validation set by early stopping.

The evaluator loops over the configured test manifests and computes global metrics over all samples. If the manifest contains a grouping field requested by the configuration, the

evaluator also computes grouped metrics. This is how the same evaluation code supports category-level reporting for ExDark and illumination-level reporting for MILLS.

The evaluator automatically adapts to paired and unpaired datasets. If a target image is available in the preprocessed CSV manifest, full-reference metrics are computed. These include SSIM, PSNR, and MAE. If the target path is empty, full-reference metrics are skipped. No-reference metrics and diagnostic metrics are computed only from the enhanced output image and can therefore be used for both paired and unpaired datasets.

The no-reference metrics NIQE and BRISQUE are not implemented explicitly. They are computed through the `pyiqa` library, using the corresponding metric objects provided by the library.

Diagnostic metrics are computed on the enhanced output $\hat{I}$. The output luminance map $\hat{Y}$ is computed with the same BT.709 definition introduced during offline preprocessing in Eq. 1, but it is applied to the model output rather than to the original input image. From this map, the framework derives three output-level summaries: mean luminance, luminance standard deviation, and dark pixel ratio. Mean luminance and dark pixel ratio help detect residual darkness or excessive brightening, while luminance standard deviation provides a compact indication of contrast variation. The dark pixel ratio uses the same threshold-based definition in Eq. 2, again applied to $\hat{Y}$.

The bright clipping ratio is used to detect possible over-exposure. It measures the fraction of pixels for which at least one RGB channel is close to saturation:

$$\text{BCR} = \frac{1}{HW} \sum_{i=1}^H \sum_{j=1}^W 1 \left[\max_{c \in \{R, G, B\}} \hat{I}_{c,i,j} > 0.98 \right]. \quad (7)$$

Together with mean luminance, this metric helps distinguish globally bright outputs from outputs that contain localized saturated regions.

The RGB channel imbalance is used as a simple indicator of color cast. It compares the mean values of the three output channels and measures the gap between the largest and smallest one:

$$\text{RGBI} = \max_{c \in \{R, G, B\}} \mu_c - \min_{c \in \{R, G, B\}} \mu_c, \quad (8)$$

where μ_c is the mean value of output channel c . A larger value indicates a stronger imbalance between color channels.

Finally, Laplacian sharpness is used as a simple measure of high-frequency content in the enhanced output. The Laplacian filter highlights rapid changes in the luminance map, such as edges, fine details, noise, or halo artifacts. The metric is the variance of the image obtained after applying the Laplacian kernel:

$$\text{LS} = \text{Var}(K * \hat{Y}), \quad (9)$$

where $*$ denotes convolution and

$$K = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}. \quad (10)$$

A very low value may indicate over-smoothing or blur, especially when luminance standard deviation is also low. An unusually high value may indicate residual noise, overly sharp edges, or halo artifacts. For halo artifacts, this signal is interpreted together with bright clipping ratio, no-reference quality scores, and visual inspection near light sources and strong edges.

During test evaluation, the framework also logs per-sample metric events. This is useful because global scores summarize average behavior, while individual sample records can later guide qualitative analysis and help identify potential failure cases.

4.4. Log Parsing and Aggregation

4.5. Qualitative Example Export

5. U-Net Family

The first model family implemented in this project is U-Net [6]. In this work, U-Net is used as a configurable encoder-decoder architecture for paired low-light image enhancement. This choice naturally matches the image-to-image nature of the task: the model receives a low-light RGB image and must produce an enhanced RGB image at the corresponding spatial resolution. Although U-Net was originally introduced for biomedical image segmentation, its encoder-decoder structure with skip connections is also well suited to this setting, because it allows the model to combine global contextual information with the preservation of local image details. The corresponding grid search explores multiple configurations by varying both architectural and optimization choices.

5.1. Architecture

U-Net is an encoder-decoder architecture with skip connections. The encoder progressively reduces the spatial resolution of the image and extracts increasingly abstract features, while the decoder restores the spatial resolution and produces the enhanced RGB image. The deepest part of the network is called the bottleneck: it is the point where the feature maps have the lowest spatial resolution but the richest semantic representation. Skip connections transfer encoder features to decoder stages at matching resolutions, except at this bottleneck level, where no skip connection is needed. This helps the decoder recover spatial details that may be weakened during downsampling, while still using the contextual information extracted by the deeper layers.

The implemented U-Net is built from repeated convolutional blocks. A convolutional layer learns local filters over

the image or over intermediate feature maps. A feature map is an internal representation produced by the network: it can be seen as a set of channels that store the patterns detected at a given layer, such as edges, textures, colors, or more abstract structures. In this implementation, each block applies two convolutional layers, and each convolution is followed by two possible operations: normalization, when enabled, and a non-linear activation function.

In each convolutional block, the convolutions use a 3×3 kernel with padding equal to one. This choice preserves the spatial resolution inside the block, so that feature maps are not progressively reduced by the convolutions themselves. Spatial resolution changes are therefore confined to the explicit downsampling and upsampling stages. This is useful for paired image enhancement, where preserving spatial correspondence between the prediction and the reference target is important for pixel-wise supervision.

The normalization layer is optional, and different normalization strategies are part of the search space. Normalization can make training more stable by controlling the distribution of intermediate feature values. The activation function is instead kept fixed to leaky rectified linear unit (LeakyReLU) in the U-Net experiments. This function plays the same general role as a standard rectified linear unit (ReLU), because it introduces non-linearity and allows the network to learn transformations that are more complex than a simple linear filter. However, unlike ReLU, it does not completely suppress negative activations, but keeps a small non-zero response for them. This choice was preferred to reduce the risk of inactive units and to maintain a more stable gradient flow during training.

Downsampling is performed with 2×2 max pooling followed by a convolutional block. Max pooling reduces the spatial size of the feature maps, allowing deeper layers to process a larger image context. Upsampling is performed either with bilinear interpolation or with transposed convolution, depending on the configuration. Bilinear interpolation is a fixed, non-learnable resizing operation. Transposed convolution is instead learnable, so the model can adapt how feature maps are upsampled.

The final layer is a 1×1 convolution followed by a sigmoid function. The 1×1 convolution maps the decoder features to the three RGB output channels. The sigmoid function constrains the output values to the $[0, 1]$ range, matching the normalized image representation used throughout the pipeline.

5.2. Training Objective and Grid Search

The U-Net family follows the general experimental protocol described in Section 3. For this family, training uses a combined L1-SSIM objective. The L1 term penalizes pixel-wise differences between the enhanced output and the reference image, using the MAE defined in Eq. 3. The structural term

is implemented as an SSIM-based penalty, using the SSIM definition in Eq. 4. Since SSIM is a similarity score to maximize, it is converted into a loss term by using $1 - \text{SSIM}$, which becomes smaller when the prediction is structurally closer to the reference image. The loss is defined as

$$\mathcal{L}(\hat{I}, I) = 0.8 \text{MAE}(\hat{I}, I) + 0.2 (1 - \text{SSIM}(\hat{I}, I)), \quad (11)$$

where $\hat{I}$ is the enhanced output and I is the reference image. The two coefficients are configurable in the experiment file, but they are kept fixed in the grid search.

The grid search defined for this family explores a set of architectural and optimization choices, reported in Table 3. The architectural part of the grid varies the channel schedule, the normalization strategy, and the upsampling method. The channel schedule specifies how many feature channels are used at each resolution level of the U-Net. As the encoder goes deeper, the spatial resolution decreases and the number of channels usually increases, allowing the network to represent progressively richer features. Therefore, a smaller channel schedule defines a lighter model with fewer parameters, while a larger schedule increases model capacity at the cost of higher computational complexity.

The normalization choice determines how intermediate feature activations are normalized inside the convolutional blocks. The search compares no normalization, batch normalization, instance normalization, and group normalization. These alternatives differ in the statistics used for normalization, respectively relying on mini-batch statistics, per-image statistics, or groups of channels. For group normalization, the number of groups is fixed and is not part of the grid.

The upsampling method controls how decoder feature maps are brought back to higher spatial resolutions. Transposed convolution is a learnable upsampling operation and represents a standard choice in encoder-decoder architectures. However, as discussed by Odena et al. in *Deconvolution and Checkerboard Artifacts* [4], learned upsampling operators such as transposed convolutions may introduce visible periodic artifacts when not properly controlled. For this reason, the grid also includes bilinear interpolation, which performs a fixed non-learnable resize before the following convolutional block. This alternative is less flexible, but provides a simpler upsampling mechanism and can reduce the risk of upsampling artifacts.

Finally, the grid varies two optimization hyperparameters: the learning rate and the weight decay. The learning rate controls the magnitude of the optimizer updates, while weight decay acts as a regularization term on the model weights.

Combining these choices gives 144 U-Net configurations in the single-seed screening stage.

6. Conclusion

...

Table 3. Hyperparameters explored in the U-Net grid search.

Hyperparameter	Values
Channel schedule	(16, 32, 64, 128), (32, 64, 128, 256), (32, 64, 128, 256, 512)
Normalization	None, batch, instance, group
Upsampling	Bilinear, transposed
Learning rate	$3 \cdot 10^{-4}$, $5 \cdot 10^{-4}$, 10^{-3}
Weight decay	0, 10^{-4}

References

- [1] Yuen Peng Loh and Chee Seng Chan. Getting to know low-light images with the exclusively dark dataset. *Comput. Vis. Image Underst.*, 178:30–42, 2019. [1](#)
- [2] Anish Mittal, Anush Krishna Moorthy, and Alan Conrad Bovik. No-reference image quality assessment in the spatial domain. *IEEE Trans. Image Process.*, 21(12):4695–4708, 2012. [4](#)
- [3] Anish Mittal, Rajiv Soundararajan, and Alan C. Bovik. Making a “completely blind” image quality analyzer. *IEEE Signal Process. Lett.*, 20(3):209–212, 2013. [4](#)
- [4] Augustus Odena, Vincent Dumoulin, and Chris Olah. Deconvolution and checkerboard artifacts. *Distill*, 2016. [10](#)
- [5] Maria Pilligua, David Serrano-Lozano, Pai Peng, Ramon Baldrich, Michael S. Brown, and Javier Vazquez-Corral. Evaluating low-light image enhancement across multiple intensity levels. *CoRR*, abs/2511.15496, 2025. [1](#)
- [6] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *MICCAI (3)*, pages 234–241. Springer, 2015. [9](#)
- [7] Zhou Wang, Alan C. Bovik, Hamid R. Sheikh, and Eero P. Simoncelli. Image quality assessment: from error visibility to structural similarity. *IEEE Trans. Image Process.*, 13(4): 600–612, 2004. [3](#)
- [8] Chen Wei, Wenjing Wang, Wenhan Yang, and Jiaying Liu. Deep retinex decomposition for low-light enhancement. In *BMVC*, page 155. BMVA Press, 2018. [1](#)
- [9] Wenhan Yang, Wenjing Wang, Haofeng Huang, Shiqi Wang, and Jiaying Liu. Sparse gradient regularized deep retinex network for robust low-light image enhancement. *IEEE Trans. Image Process.*, 30:2072–2086, 2021. [1](#)